

2400-DS1AL # Algorithms for Data Science

[Kokpit](#) / [Moje kursy](#) / [2400-DS1AL#2025L#277553](#) / [Lecture 8 & 9](#) / [Lecture notes](#)

Lecture notes

Credits: dr Krzysztof Fleszar

Lecture 6: Dynamic Programming

Dynamic programming is a method of constructing efficient algorithms that tends to be especially useful in counting and optimization problems.

We start with a very simple example. The Fibonacci sequence is given by $F_1=1, F_2=1, F_{n+2}=F_n+F_{n+1}$ (in a different definition, the sequence starts with $F_0=0$). How to compute the n -th Fibonacci number?

INPUT: positive integer n

OUTPUT: F_n

We could compute F_n with the following recursive program (this method is also called backtracking):

```
def fib(n):  
    if n <= 2:  
        return 1  
    else:  
        return fib(n-2) + fib(n-1)
```

What is the running time of this algorithm? The value of F_5 will be computed in the following way: $F_5 = F_3 + F_4 = (F_1 + F_2) + (F_2 + F_3) = (1+1) + (1+(F_1 + F_2)) = (1+1) + (1+(1+1))$.

As we can see, our algorithm basically adds F_n many ones; thus, the running time will be proportional to F_n . It is known that $F_n = \Theta(\varphi^n)$ where $\varphi = 1 + 5\sqrt{2} = 1.618\dots$ (The constant φ is also known as the *golden ratio*.) Thus the running time is exponential which is bad. The memory complexity is $O(n)$, because of the recursion stack (for a brief description of recursion stacks, see our discussion of quick sort).

However, this algorithm can be easily improved. Note how we have computed F_3 twice in our situation—we could save time by computing each F_k just once, as shown in the following Python and C++ codes:

```
def fib(n):
    if n <= 2:
        return 1
    fibs = [0] * (n+1)
    fibs[1] = 1
    fibs[2] = 1
    for i in range(3, n+1):
        fibs[i] = fibs[i-1] + fibs[i-2]
    return fibs[n]
```

```
int fib(int n) {
    vector<int> fibs(n+1);
    fibs[1] = fibs[2] = 1;
    for(int i=3; i<=n; i++) fibs[i] = fibs[i-2] + fibs[i-1];
    return fibs[n];
}
```

It is easy to see that now our algorithm runs in linear time, that is, in time $\Theta(n)$. This is a tremendous speed-up with respect to our previous approach! *To get a feeling for the speed-up, just run the codes for various values of n !* The memory of the second approach is also $\Theta(n)$. Note that we never use position 0 of our array. But even if we would decrease the size of our array by one (for example, by shifting everything to the left in the sense that $\text{fibs}[i]$ would mean F_{i+1}), the used memory would not change asymptotically.

Interestingly, we can improve our algorithm even further—we only ever need the last two computed Fibonacci numbers:

```
def fib(n):  
    a = 1  
    b = 1  
    # a is the next-to-last and  
    # b the last computed Fibonacci number.  
    for _ in range(n-2):  
        (a,b) = (b, a+b)  
    return b
```

Now the memory is just $O(1)$.

This kind of optimization is called *dynamic programming*, abbreviated by *DP*. The high level idea is that we fill a (possible multi-dimensional) table from one side to the other where each entry corresponds to a solution to a subproblem. The table is organized such that every entry can be computed by using only previously computed entries. One of the final entries contains a solution to the overall problem.

In more detail, the approach works as follows:

- Start with a (typically very inefficient) recursive solution.
- Identify the subproblems which are computed multiple times by our recursive solution. (In the Fibonacci example, computing $\text{fib}(k)$ is such a subproblem.)
- Determine a natural ordering of the subproblems where a subproblem depends only on previous subproblems. The last subproblems will typically contain the solution to the overall problem. (In our example, such an ordering is $\text{fib}(1), \text{fib}(2), \dots, \text{fib}(n)$).
- Create an empty table with an entry for each subproblem.
- Directly solve the first subproblems that do not depend on any other subproblem. Write their solutions into the table. (In our example, we store $\text{fib}(1)$ and $\text{fib}(2)$.)
- Solve the remaining subproblems according to the ordering by using the table to look up any needed solutions of previous subproblems, and each time write the result into the table. (In our example, to compute $\text{fib}(i+2)$, we look up the solutions to $\text{fib}(i+1)$ and $\text{fib}(i)$.)

Thus, dynamic programming typically is iterative and bottom-up instead of recursive and top-down.

Note: The times stated above are actually the number of basic arithmetic operations. Though the stated times hold in our RAM model (with unbounded memory cell size), this is not too realistic as the Fibonacci numbers are extremely big and, in reality (and in more sophisticated computer models), they wouldn't fit into single memory cells. In general, a value of size M occupies $\Theta(\log M)$ bits. Thus, the n -th Fibonacci number

uses as many as $\Theta(\log Fn) = \Theta(\log \varphi n) = \Theta(n \log \varphi) = \Theta(n)$ bits! Consequently, just to write Fn down realistically takes linear time in n . Thus, to be more realistic, the time and space complexities above have to be multiplied by n . For example, our dynamic programming algorithm would realistically take $\Theta(n^2)$ time and $\Theta(n)$ space.

So, is the DP algorithm above optimal? No, we can do it better! It is possible to compute Fn (and to solve linear recurrences in general) using just $O(\log n)$ basic arithmetic operations instead of $\Omega(n)$ many—with a different approach than above. On the exercise sheet, you find a task about it.

The Fibonacci example was very simple. Before considering more complex problems, we first consider another approach to solve recursive optimization problems.

Recursion with Memoization

DP is one possible approach of optimizing recursive computations. There is another approach, called Recursion with Memoization, where we modify our backtracking solution in the following way:

1. once we compute our answer, we record it somewhere;
2. before we start computing, we check our notes to see if we have already computed the answer—if yes, just use the computed value.

In Python, we can use a global dictionary to record the answers (we learn more about dictionaries in the next lecture):

```
memo = {}
def fib(n):
    global memo
    if n in memo:
        return memo[n]
    if n <= 2:
        answer = 1
    else:
        answer = fib(n-2) + fib(n-1)
    memo[n] = answer
    return answer
```

The command `n in memo` checks if memo contains any value on "position" `n` (to be more precise: "with key `n`"—see next lecture).

The space complexity is $\Theta(n)$ as we store about n values. What about the running time? It depends on the total number of recursive calls of our function as well as on the access time to the dictionary. For every value of n , only the first call to $\text{fib}(n)$ leads to further recursive calls (at most two), all subsequent calls to $\text{fib}(n)$ do not spread further thanks to memoization. Hence, in total over all executions for a given n , our function makes at most two recursive calls! This means that, in total over all values of n , our function is called not more than $2n$ times by itself! Thus, if the time to access our dictionary is $f(n)$, the total running time of our approach is $\Theta(n \cdot f(n))$.

In Python, the (worst case) access time to a dictionary is guaranteed to be in $O(n)$. Thus, the algorithm above runs in $O(n^2)$ time which is extremely faster than the $\Theta(\varphi^n)$ time of recursion without memoization. But is it as fast as the $\Theta(n)$ time of the DP approach? The surprising answer is: in practice yes, there should be no big difference. The reason is that in Python the *average* access time to a dictionary is just $O(1)$ which implies that our algorithm runs in $\Theta(n)$ *average* time! In only very rare (or artificial) cases the running time would be higher.

Nonetheless, by using an array instead of a dictionary, we can bring the worst case time even down to $\Theta(n)$. However, in general this is not always a good idea. There are optimization problems where it is more efficient to use a dictionary despite its higher access times (as a dictionary stores only the values we need); there are even problems where using a simple array is not sufficient. Furthermore, it is easier to use an (existing) dictionary as the one of Python, and then also the code becomes simpler; e.g., we don't need to bother on how to simulate empty entries as we would need if we had an array. However, we have to keep in mind that dictionaries are rather complex data structures as we will see in the next lecture.

Comparison of DP and Memoization

The two approaches, Dynamic Programming and Recursion with Memoization, are largely interchangeable, but there are significant differences:

- The idea of memoization might be easier to grasp by some people / for some problems.
- In some problems it is not clear how to list, sequence, and index all the subproblems which could appear in our computation (without missing any necessary ones, and without solving too many subproblems which are not actually necessary). Recursion with memoization avoids this issue completely.
- Filling arrays is easier for the computer than doing recursion and filling dictionaries. For this reason, DP tends to be faster and more memory efficient than memoization. On top of this, memoization also tends to have worse usage of memory because of the recursion stack.
- In some settings, the depth of recursive calls is limited. For example, Python by default does not allow us to have a recursion depth of more than 1000 (to prevent buggy recursive programs from using all the system memory), and thus it will fail if we try to compute, say, F_{3000} . (In Python this problem can be circumvented by using `sys.setrecursionlimit(x)` to increase the depth limit to a value x .)

Knapsack Problem

The Knapsack Problem is a very natural optimization problem. You have a bunch of valuable items you could take, but you cannot take them all, because of limited storage space. Which subset of items should you take in order to maximize the value of your subset?

INPUT: n items numbered from 0 to $n-1$, for every item, a weight w_i and value v_i ; capacity M

OUTPUT: A subset S of $\{0..n-1\}$ such $\sum_{i \in S} w_i \leq M$ and $\sum_{i \in S} v_i$ is maximized

We assume that the weights $w_0, \dots, w_{n-1}$ and the capacity M are integers. A simple solution to the Knapsack problem is to take the item which has the greatest value-to-weight ratio among the items that still fit in our storage space, and then fill the remaining storage using the same method. This is called the greedy approach (always start with the move which looks the most promising given some incomplete information). There are many problems where the greedy approach works, but not here: if the capacity is 100, the weights are $[50, 50, 51]$ and the values are $[50, 50, 52]$, this approach would make us take the item with value 52, while we could also fit the two items with value 50, for almost twice as good total value!

To apply the dynamic programming approach, we need to define subproblems. Let's decide first whether we want to take the last item (numbered $n-1$) or not. If we take it, we need to fill the remaining space ($M - w_{n-1}$) optimally with items with smaller numbers. If we do not take it, we need to fill the space M optimally with items with smaller numbers. Therefore, the appropriate subproblem is an instance of the same problem where we have less items to choose, and possibly less carrying capacity.

Let $B_{m,k}$ be the value of a best solution to the knapsack problem when we restrict our capacity to m and our items to $0..k-1$. Since we can either take the item $k-1$ or not, we get the following:

- $B_{m,0}=0$ for every m ,
- $B_{m,k}=B_{m,k-1}$ if $m < w_{k-1}$ (the $k-1$ -th element does not fit), and
- $B_{m,k}=\max(B_{m,k-1}, B_{m-w_{k-1},k-1}+v_{k-1})$ otherwise.

To find the best possible value, we need to compute $B_{M,n}$. Computing all the values of $B_{m,k}$ yields an algorithm which runs in time $\Theta(Mn)$ and memory $\Theta(Mn)$. In our algorithm, we treat B as a two-dimensional array $B[0..M][0..n]$. In dynamic programming, such an array is called a *DP table*.

Once we have computed the optimal value, the actual set can be reconstructed by checking how we have achieved BM,n , that is, whether we took the $(n-1)$ -th element or not. If we didn't take the element, we repeat the question for $BM,n-1$. If we took the element, we repeat the question for $BM-n-1,n-1$. We repeat the whole process until we reach $Bm,0$ for some m .

If we care only about the best value, we can reduce the memory complexity to $\Theta(M)$ since we only use $Bm,k-1$ when computing Bm,k . Thus, after having computed Bm,k for all values of m , we can discard $Bm,k-1$ for every value of m from our memory.

Above, we could either take an item or not. A similar algorithm can be used in the case when there are many copies of every item, and we can take as many as we want.

On the slides, you find several algorithms in Pseudocode that solve the Knapsack problem.

[Here](#) you find the algorithm for the Knapsack problem in Python for all variants that we discussed:

- the two greedy variants,
- backtracking,
- recursion with memoization,
- dynamic programming as well as the optimization of reducing the space, including the reconstruction of the actual solution.

Longest Common Subsequence (LCS)

INPUT: Two sequences of integers $a[0..n-1]$ and $b[0..m-1]$

OUTPUT: A sequence of integers $c[0..k-1]$ which is the longest subsequence of both a and b . (In case of a tie, any longest subsequence can be returned.)

For example, for $a=[1,2,3,4,5,6,7,8]$ and $b=[1,4,7,2,5,8,3,6]$, the longest common subsequence is $[1,2,5,6]$. This problem has important applications in computational biology (comparing genomes).

How to solve this? For a sequence s , let s^* be its last element, and s' be the subsequence of the remaining ones; thus $s=s' s^*$. If c is a common subsequence of a and b , one of the following always holds:

- a , b , and c are all empty.
- a^* is not used in c , thus c is a common subsequence of a' and b .
- b^* is not used in c , thus c is a common subsequence of a and b' .

- a^* and b^* are in c , that is, $c^*=a^*=b^*$ (the last elements of c , a and b are equal) and c' is a common subsequence of a' and b' .

It is easy to see (or to prove by induction) that we can therefore compute the LCS of a and b recursively. The LCS of a and b has to be equal to one of the following:

- an empty sequence,
- the LCS of a' and b , or
- the LCS of a and b' , or
- if $a^*=b^*$, the LCS of a' and b' plus the a^* (equivalently b^*) appended.

We always take the option which gives us the longest common subsequence.

A recursive solution will run in exponential time, but we can see that every subproblem boils down to computing the LCS of $a[0..i-1]$ and $b[0..j-1]$. We can use DP to store the results in a two-dimensional array dp at position $[i][j]$. Therefore, we can compute the length of the LCS using the following DP program (see [full Python code](#) with reconstruction):

```
dp = np.zeros([n+1, m+1])
for i in range(1, n+1):
    for j in range(1, m+1):
        if a[i-1] == b[j-1]:
            dp[i][j] = dp[i-1][j-1] + 1
        else:
            dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

At the end, $dp[n][m]$ contains the length of the LCS of a and b . As in the Knapsack problem, once having computed the length of the LCS, we can use the whole dp table to retrieve the actual LCS by checking how we have achieved $dp[n][m]$, and going back to $dp[0][0]$. See [this visualization](#) (C++ source is also included).

This program runs in time $\Theta(nm)$ and memory $\Theta(nm)$. If we are just interested in the length of the LCS, we can improve the memory complexity to $\Theta(m)$ (or $\Theta(n)$) by remembering only the last two rows of the array dp .

The remaining part of this chapter is not mandatory for the course, but it might help you to better understand dynamic programming.

Matrix Chain Multiplication

We are given a chain of matrices: A_0 of dimensions $d_0 \times d_1$, A_1 of dimensions $d_1 \times d_2$, ..., A_{n-1} of dimensions $d_{n-1} \times d_n$. We can compute the product of two matrices of dimensions $a \times b$ and $b \times c$ using the brute force algorithm that needs $a \cdot b \cdot c$ many "normal" multiplications (i.e., involving only numbers). Since matrix multiplication is commutative, we can multiply them in any order we want—what ordering should we use to minimize the number of number multiplications?

Let $t(i, j)$ be the number of "normal" multiplications necessary to obtain $A_i \dots A_{j-1}$. Suppose the last multiplication will multiply $(A_i \dots A_{k-1})$ by $(A_k \dots A_{j-1})$; the smallest number of "normal" multiplications possible by using this method is $t(i, k) + t(k, j) + d[i]d[k]d[j]$. Again, we could compute $t(i, k)$ and $t(k, j)$ recursively (in exponential time), or use DP to avoid recomputing the same values of $t(i, j)$ over and over again. This time, it is a bit more challenging to write a program which computes the values $t(i, j)$ of our dp table in the correct order (i.e., by using only values which we have already computed)—we can do this e.g. like this (see [full code](#) online):

```
t = np.zeros((n+1, n+1))

for j in range(n):
    for i in range(j-1, -1, -1):
        if j==i+1:
            t[i][j] = 0
        else:
            t[i][j] = min([t[i][k]+t[k][j]+d[i]*d[j]*d[k] for k in range(i+1, j)])
```

The code works in time $\Theta(n^3)$ and memory $\Theta(n^2)$. (In contrast to the problems above, it is not possible to reduce the memory complexity in any easy way, since usually all the computed values will still be necessary later.) See [this visualization \(C++ source is also included\)](#).

Ostatnia modyfikacja: wtorek, 2 czerwca 2026, 22:20

◀ [Lecture 9 - recording](#)

Przejdź do...

[Lecture 10](#) ▶

✉ Skontaktuj się z pomocą techniczną

Jesteś zalogowany(a) jako Ondřej Marvan (Wyloguj)
2400-DS1AL#2025L#277553